

DeskBuddy System

Projektdokumentation

PROG3 & ADP · HF Informatik, gibb Bern 2026 · Venurshan Manivannan

1. Einleitung

DeskBuddy ist ein kleiner Schreibtisch-Assistent, der Termine aus Google Calendar anzeigt. Statt den ganzen Kalender zu zeigen, zeigt er nur den aktuell laufenden und den nächsten Termin. Die Anzeige läuft auf einem kleinen ESP32-Gerät mit Display und zusätzlich in einem Web-Dashboard.

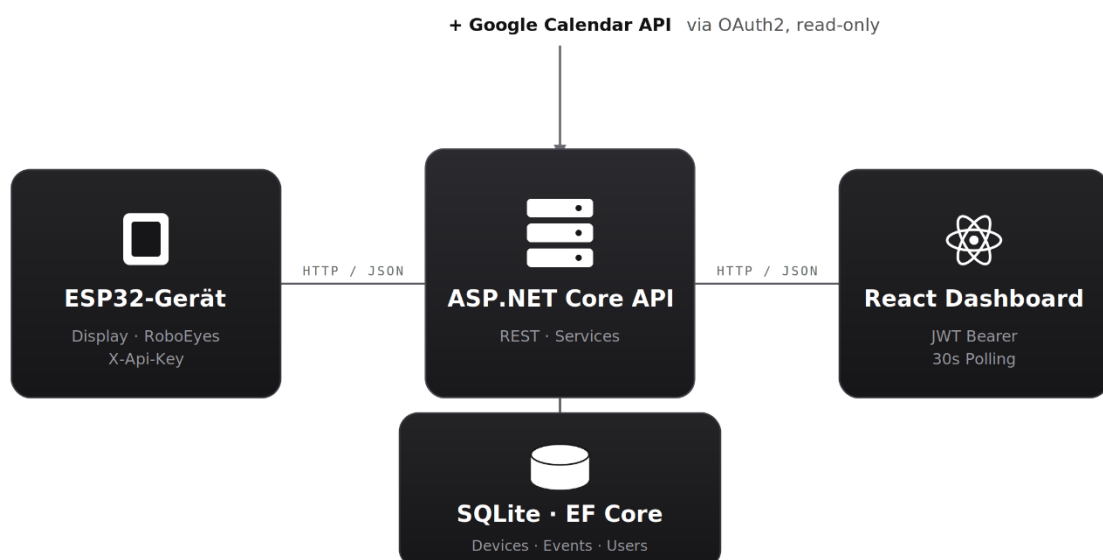
Das Projekt deckt zwei Module ab. In PROG3 geht es um die technische Umsetzung: API, Datenbank, Authentifizierung, Frontend und Tests. In ADP geht es um den Arbeitsprozess: Sprints, ein Kanban-Board, Akzeptanzkriterien und Reviews.

2. Systemarchitektur

Das System besteht aus drei Teilen: den Clients (ESP32-Gerät und React-Dashboard), dem Backend (ASP.NET Core API mit SQLite-Datenbank) und der externen Google Calendar API. Alle Teile reden über HTTP mit JSON miteinander. Das Backend ist das Herzstück: dort laufen alle Daten zusammen.

Die beiden Clients melden sich unterschiedlich an, weil ein Mensch und ein Gerät verschiedene Anmeldungen brauchen:

Verbindung	Protokoll	Authentifizierung
ESP32-Gerät → API	HTTP / JSON	API-Key (X-API-Key)
Dashboard → API	HTTP / JSON	JWT Bearer Token
API → SQLite	EF Core	lokal, kein Netzwerk
API → Google Calendar	REST	OAuth2 (nur lesen)



3. Backend (ASP.NET Core)

Das Backend ist eine ASP.NET Core 8 Web API. Die Daten werden mit Entity Framework Core in einer SQLite-Datenbank gespeichert. SQLite reicht hier aus, weil das System lokal für ein einzelnes Gerät läuft und keine grosse Datenmenge anfällt. Über EF Core wäre ein späterer Wechsel auf eine andere Datenbank ohne grossen Aufwand möglich.

3.1 Datenmodell und Persistenz

Es gibt drei Tabellen: Device (das Gerät), CalendarEvent (ein Termin) und User (für den Login). Die Tabellen sind bewusst nicht über Fremdschlüssel verbunden, da das Gerät unabhängig von den Kalenderdaten arbeitet. Die Datenbank wird über zwei EF-Core-Migrationen aufgebaut.

3.2 CRUD und DTOs

Für zwei Objekte gibt es vollständiges CRUD (Anlegen, Lesen, Ändern, Löschen): Device und CalendarEvent, mit je fünf Endpunkten. Zwischen den internen Modellen und der API liegt eine DTO-Schicht mit zehn DTOs. Dadurch wird nur das nach aussen gegeben, was wirklich gebraucht wird. Beispiel: Der API-Key eines Geräts ist intern gespeichert, taucht aber in keiner Antwort der API auf. Falsche Eingaben werden über Validierung abgefangen und mit Statuscode 400 und einer passenden Fehlermeldung beantwortet.

3.3 Authentifizierung

Es gibt zwei Wege, weil ein Gerät keinen Login-Bildschirm hat. Menschen melden sich am Dashboard mit Benutzername und Passwort an und bekommen ein JWT (acht Stunden gültig). Passwörter werden mit BCrypt gehasht. Das Gerät meldet sich mit einem festen API-Key im Header X-API-Key an. Kurz gesagt: JWT für Menschen, API-Key für Maschinen.

3.4 Google Calendar und Hintergrund-Sync

Das Backend holt die Termine über OAuth2 (nur Lesezugriff) von Google Calendar. Der Sync läuft automatisch alle fünf Minuten über einen Hintergrunddienst und kann zusätzlich von Hand ausgelöst werden. Beim Sync werden alle alten Termine gelöscht und neu eingefügt (Full-Replace). Das ist einfacher und vermeidet Probleme mit doppelten oder veralteten Einträgen.

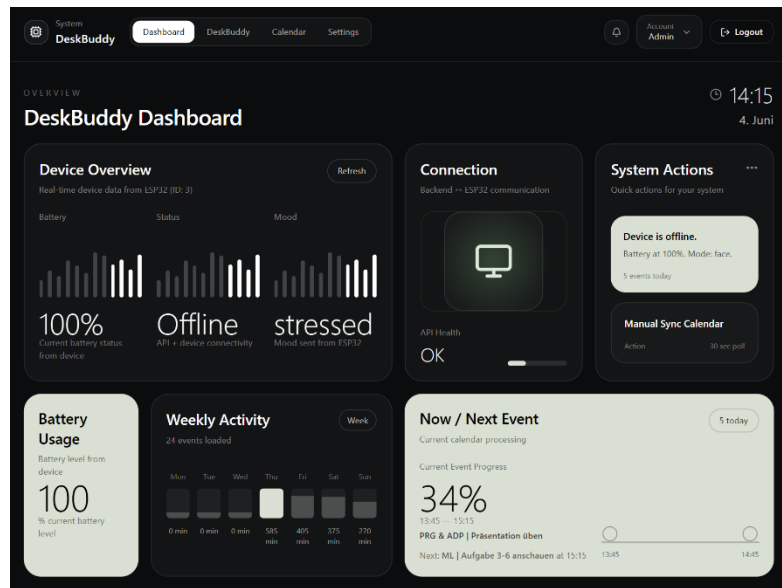
3.5 Now/Next-Logik

Welcher Termin gerade läuft und welcher als nächstes kommt, berechnet die Klasse NowNextCalculator. Sie enthält reine Logik ohne Datenbankzugriff und lässt sich dadurch gut mit Unit-Tests prüfen. „Now“ ist der Termin, der gerade läuft; „Next“ ist der nächste zukünftige Termin. Beide können leer sein, wenn gerade kein passender Termin existiert.

4. Frontend (React Dashboard)

Das Dashboard ist mit React 19, Vite und Tailwind CSS gebaut und nutzt ein dunkles Design. Es hat fünf Seiten:

- Login: Anmeldung mit Fehlermeldung bei falschen Daten.
- Dashboard: Gerätestatus, Wochenaktivität und eine Now/Next-Karte.
- DeskBuddy: acht Stimmungs-Animationen (per CSS), die zum Gerät passen.
- Kalender: Monatsansicht, anstehende Termine und ein Sync-Knopf.
- Einstellungen: System-Infos und Geräte-ID.



Alle Anfragen laufen über einen zentralen API-Client. Das JWT wird gespeichert und bei jeder Anfrage mitgeschickt; bei einem abgelaufenen Token meldet sich der Client automatisch neu an. Die Daten werden alle 30 Sekunden neu geladen.

5. ESP32-Gerät

Als Gerät dient ein Waveshare ESP32-S3 mit rundem AMOLED-Touch-Display. Es verbindet sich per WLAN mit dem Backend und wird per USB betrieben. Es hat zwei Anzeigemodi: einen Face-Modus mit animierten Augen (RoboEyes) und einen Calendar-Modus mit dem aktuellen und nächsten Termin. Per Touch wird zwischen den Modi gewechselt.

6. Testing

Getestet wurde auf mehreren Ebenen, von schnellen, isolierten Tests bis zu Tests über das ganze System. Insgesamt laufen 20 automatisierte Tests, alle erfolgreich. Dazu kommen manuelle Tests. Details und Ergebnisse stehen im Anhang.

Art	Anzahl	Was geprüft wird
Unit-Tests (xUnit)	13	Now/Next-Logik, ohne Datenbank
Integrationstests (xUnit)	7	ganze API mit In-Memory-Datenbank
API-Tests (Postman)	25 / 42 Asserts	echte Endpunkte mit Authentifizierung
Manuell	12 + 7 + 4	Frontend, Gerät, End-to-End-Szenarien

Wichtig: Die automatisierten Tests laufen ohne echten Server und ohne echte Google-Verbindung. Eine In-Memory-Datenbank und ein Fake für den Google-Dienst machen das möglich. Genau das ist der Zweck der Integrationstests.

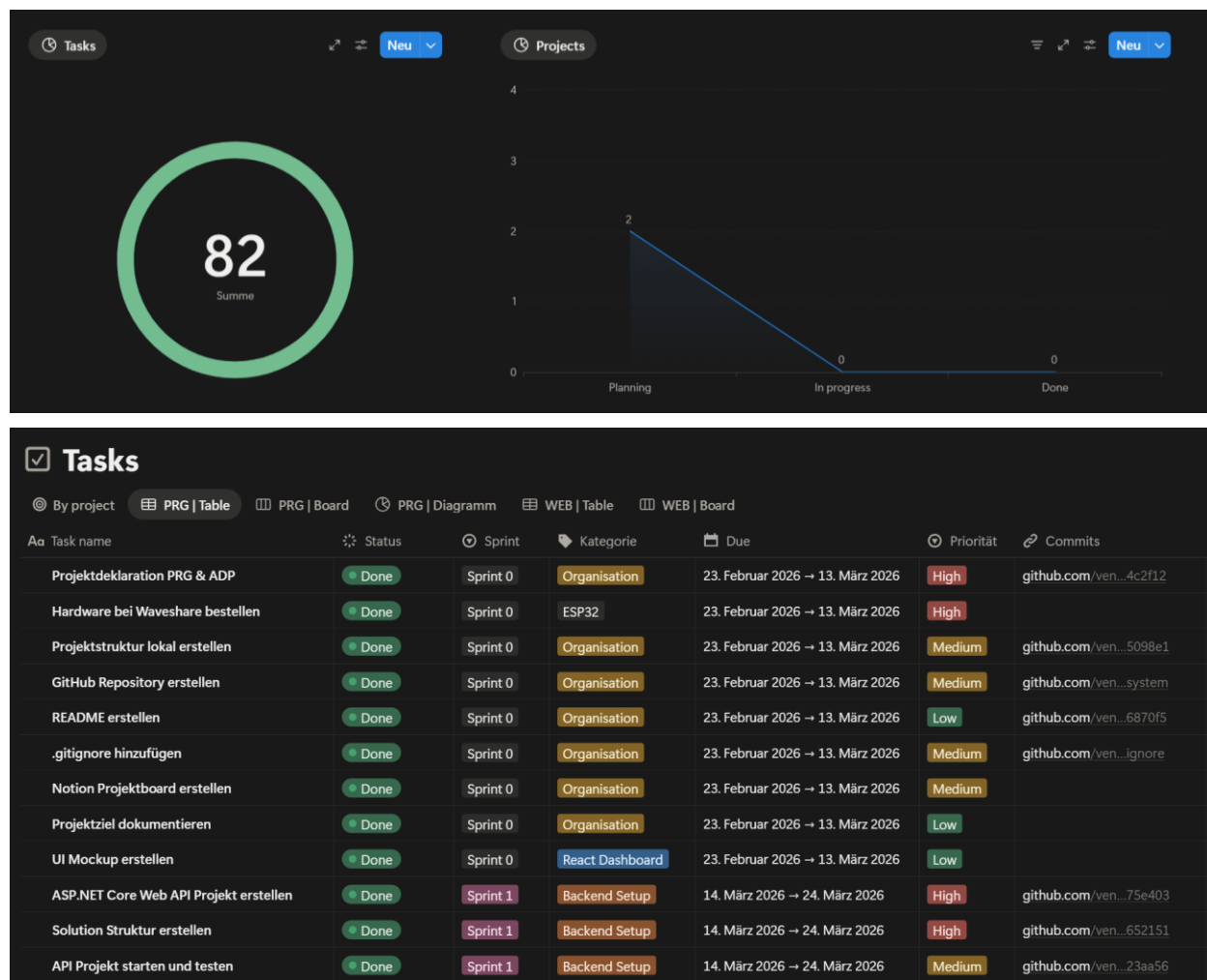
7. Sicherheit

Sensible Daten landen nicht in der Versionskontrolle. Die Google-Zugangsdaten liegen in einem Secrets-Ordner, die Datenbankdatei und der Geräte-Ordner sind über .gitignore ausgeschlossen. Passwörter werden nur als BCrypt-Hash gespeichert, und der Google-Zugriff ist auf Lesen beschränkt.

Verbesserungsvorschlag zur Firmware: Aktuell ist der ganze Geräte-Ordner ausgeschlossen, damit keine echten Zugangsdaten ins Repository gelangen. Besser wäre, eine Beispiel-Firmware mit einer Datei config.example.h einzuchecken, in der nur Platzhalter für WLAN und API-Key stehen. Die echte config.h mit den richtigen Werten bleibt weiterhin per .gitignore ausgeschlossen. So ist der Aufbau im Repository nachvollziehbar, ohne dass Geheimnisse sichtbar werden.

8. Agile Entwicklung (ADP)

Die Arbeit war in sieben Sprints (0 bis 6) aufgeteilt, jeder mit einem klaren Schwerpunkt. Verwaltet wurden die Aufgaben auf einem Notion-Kanban-Board mit den Spalten To Do, In Progress und Done. Insgesamt gab es 82 Aufgaben, die am Ende alle abgeschlossen waren (100 %).



Jede Aufgabe hatte Akzeptanzkriterien, zum Beispiel „Login liefert bei korrekten Daten ein Token“ oder „Gerät sendet alle 30 Sekunden einen Heartbeat“. Die Commits in GitHub wurden mit den Aufgaben im Board verknüpft, sodass man zu jeder Aufgabe die passenden Code-Änderungen findet. Nach jedem Sprint gab es ein kurzes Review, in dem der Stand geprüft und das weitere Vorgehen angepasst wurde.

9. Reflexion und bekannte Einschränkungen

Gut funktioniert hat die Trennung der Now/Next-Logik in eine eigene Klasse, weil sie sich dadurch sehr einfach testen liess. Auch die Anbindung an Google Calendar und das Gerät liefen nach der Einrichtung stabil. Die grösste Hürde war der Sync mit der Datenbank: Ein aktualisierender Ansatz führte zu Konflikten in EF Core, gelöst wurde das mit dem Full-Replace-Muster. Auch der einmalige OAuth-Login im Browser war etwas umständlich.

Ehrlich benannte Einschränkungen:

- Das System läuft nur lokal, ohne Cloud-Deployment und ohne CI/CD.
- Die Geräte-ID ist im Frontend fest hinterlegt; es gibt keine Geräteauswahl.
- Der Batteriestand ist fest auf 100 gesetzt, da das Gerät per USB läuft und keinen Akkusensor hat.
- Die Admin-Zugangsdaten liegen in der appsettings.json und sind nur für die Entwicklung gedacht.
- Es gibt kein automatisiertes Frontend-Testing.
- Der Google-Zugriff braucht einen einmaligen manuellen Browser-Login.

Mögliche nächste Schritte wären ein Cloud-Deployment, die Unterstützung mehrerer Geräte mit Auswahl im Frontend, das Auslagern der Zugangsdaten und ein automatisiertes Frontend-Testing.

Anhang

A. Vollständige API-Endpunkte

Methode	Endpoint	Auth	Beschreibung
GET	/api/health	—	Health-Check
POST	/api/auth/login	—	Token erhalten
GET	/api/auth/me	JWT	Aktuellen Benutzer abrufen
GET	/api/devices	JWT	Geräte auflisten
POST	/api/devices	JWT	Gerät anlegen
PUT	/api/devices/{id}	JWT	Gerät ändern
DELETE	/api/devices/{id}	JWT	Gerät löschen
GET	/api/devices/{id}/status	JWT	Gerätestatus
POST	/api/devices/{id}/heartbeat	API-Key	Heartbeat senden
GET	/api/calendarevents	JWT	Termine auflisten
POST	/api/calendarevents	JWT	Termin anlegen
PUT	/api/calendarevents/{id}	JWT	Termin ändern
DELETE	/api/calendarevents/{id}	JWT	Termin löschen
GET	/api/googlecalendar/events	JWT	Google-Termine abrufen
POST	/api/googlecalendar/sync	JWT	Sync auslösen
GET	/api/nownext	API-Key	Aktueller/nächster Termin

B. Datenmodell und DTOs

Tabelle	Felder
Device	Id, Name, ApiKey, IsOnline, BatteryLevel, Mood, Mode, LastSeen
CalendarEvent	Id, GoogleEventId, Title, StartTime, EndTime, Location, Description, FetchedAt
User	Id, Username, PasswordHash (BCrypt)

DTO-Kategorie	DTOs
Device	DeviceCreateDto, DeviceUpdateDto, DeviceStatusDto, DeviceStatusDetailDto
CalendarEvent	CalendarEventCreateDto, CalendarEventUpdateDto, CalendarEventDto
Auth	LoginRequestDto, LoginResponseDto
Gerät / Now-Next	HeartbeatRequestDto, NowNextDto

C. Testdetails

Unit-Tests (13): prüfen die Methoden FindNow (5), FindNext (4) und IsDeviceOnline (4) inklusive Grenzfälle wie „Termin startet genau jetzt“ oder „Gerät noch nie gesehen“.

Integrationstests (7): Health-Check, Login mit gültigen und ungültigen Daten, geschützter Zugriff ohne Token, Gerät anlegen, Geräte auflisten und der Now/Next-Endpunkt – alle gegen eine In-Memory-Datenbank.

Postman (25 Requests, 42 Assertions): gegliedert in Health, Auth, Devices, CalendarEvents, GoogleCalendar und NowNext. Der Login speichert das Token automatisch für die folgenden Anfragen.

Manuell: 12 Frontend-Testfälle (Chrome), 7 Geräte-Testfälle (Serial Monitor) und 4 durchgängige End-to-End-Szenarien.